

First Python lesson

This lesson covers the basics so that you're able to use Python like a calculator (with numbers).

1. Arithmetic operators
2. Expressions
3. Arithmetic functions

We'll build the notebook up step by step. After each section, you'll solve some practice problems.

✓ Arithmetic operators

An arithmetic operator is a symbol that represents a computation with numbers.

Prompt: Show examples of basic arithmetic operators in Python

```
# Addition
print(f'Addition: 5 + 3 = {5 + 3}')
```

```
# Subtraction
print(f'Subtraction: 10 - 4 = {10 - 4}')
```

```
# Multiplication
print(f'Multiplication: 6 * 7 = {6 * 7}')
```

```
# Division
print(f'Division: 20 / 5 = {20 / 5}')
```

```
# Floor Division (removes decimals)
print(f'Floor Division: 17 // 3 = {17 // 3}')
```

```
# Modulo (remainder)
print(f'Modulo: 17 % 3 = {17 % 3}')
```

```
# Exponentiation
print(f'Exponentiation: 2 ** 3 = {2 ** 3}')
```

```
Addition: 5 + 3 = 8
Subtraction: 10 - 4 = 6
Multiplication: 6 * 7 = 42
Division: 20 / 5 = 4.0
Floor Division: 17 // 3 = 5
Modulo: 17 % 3 = 2
Exponentiation: 2 ** 3 = 8
```

Prompt: Explain "floor division" to a child.

Imagine you have 7 cookies and you want to share them with 3 friends.

In regular division, you might try to cut a cookie into pieces to make sure everyone gets exactly the same amount.

But in **floor division**, we don't like crumbs! We only give out whole cookies. So, if you give each friend 2 whole cookies, you have 1 cookie left over.

Floor division says: "Each friend gets 2 whole cookies!" It just ignores the leftovers and gives you the whole number answer.

Prompt: Now explain floor division to an adult (briefly)

In Python, floor division (`//`) performs division and then applies the floor function to the result, rounding down to the nearest integer. Unlike standard division (`/`), which returns a float, floor division returns an integer (or a float with .0 if inputs are floats) representing the largest whole number less than or equal to the quotient.

Prompt: Explain "modulo" first to a child, then to an adult (briefly).

For a Child

Imagine you have 7 cookies and you want to give them to 3 friends. You give the first friend a cookie, then the second, then the third. You keep going until everyone has the same amount.

Each friend gets 2 whole cookies, but you have **1 cookie left over** in your hand.

In math, the **modulo** is just a fancy name for that **leftover** piece! So, 7 modulo 3 is 1.

✓ For an Adult

The modulo operator (`%`) returns the remainder of a division operation. While floor division gives you the quotient, modulo provides the integer residue. For example, in the expression `17 % 3`, the result is `2` because 3 goes into 17 five times, leaving a remainder of 2.

Prompt: In some languages (like R), the caret (`^`) is also used for exponentiation. Is that so in Python?

No, in Python, the caret symbol (`^`) is the **bitwise XOR operator**. For exponentiation (raising a number to a power), Python uses the double asterisk (`**`).

- `2 ** 3` = 8 (Exponentiation)
- `2 ^ 3` = 1 (Bitwise XOR: `10` XOR `11` in binary is `01`)

```
# Demonstration
print(f'Exponentiation (2 ** 3): {2 ** 3}')
print(f'Bitwise XOR (2 ^ 3): {2 ^ 3}')
```

```
Exponentiation (2 ** 3): 8
Bitwise XOR (2 ^ 3): 1
```

****Prompt:**** Explain the expression `2^3` in Python and the result.

```
# Bitwise XOR breakdown
a = 2 # Binary: 10
b = 3 # Binary: 11
result = a ^ b

print(f'Binary of 2: {bin(a)}')
print(f'Binary of 3: {bin(b)}')
print(f'Result of 2 ^ 3: {result} (Binary: {bin(result)})')
```

```
Binary of 2: 0b10
Binary of 3: 0b11
Result of 2 ^ 3: 1 (Binary: 0b1)
```

✓ Practice: Evaluate - Print

Open the next text block, and write the result of the expression next to it: `5 + 3 = 8`

```
8 + 2 * 3
20 / 4
20 // 4
20 / 3
20 // 3
20 % 3
2 ** 4
```

Solution:

```
8 + 2 * 3 = 16
20 / 4     = 5.0
20 // 4    = 5
20 / 3     = 6.666667
20 // 3    = 6
20 % 3     = 2
2 ** 4     = 16
```

Copy the problems from the text block (as `Copy selection`) into a code block and run them.

```
8 + 2 * 3
20 / 4
20 // 4
20 / 3
20 // 3
20 % 3
2 ** 4
```

16

Did you notice that only the last expression was evaluated? How can you see all of them?

```
print(8 + 2 * 3)
print(20 / 4)
print(20 // 4)
print(20 / 3)
print(20 // 3)
print(20 % 3)
print(2 ** 4)
```

```
14
5.0
5
6.666666666666667
6
2
16
```

✓ Expressions

Prompt: Define "expression".

In Python, an **expression** is a piece of code that can be evaluated to a value. It is a combination of values (like numbers or strings), variables, operators, and calls to functions.

Whenever you type something into Python that results in a value being returned, you are using an expression.

Examples:

- `5 + 3` (evaluates to `8`)
- `len("Hello")` (evaluates to `5`)
- `x > 10` (evaluates to `True` or `False`)

✓ Practice: Run - Change

Run the examples in one code block. Fix the code if you get error messages. You can use `Explain error` if you like.

In the next code blocks:

1. Change the argument of `len()` to a number. What do you get?
2. Change `x > 10` to `x == 10` -- what do you get?

Solution:

```
print(5 + 3)
print(len("Hello"))
x = 5
print(x > 10)
```

```
8
5
False
```

```
# print(len(10))
```

```
x == 10
```

```
False
```

What you learnt in this practice:

1. Symbols like `x` (variables) need to be defined for the computer to understand them.
2. Functions like `len()` don't work for all arguments (a number does not have a `len()`, only a string does).
3. Operators `=` (assignment) and `==` (compare for equality) are different.

✓ Arithmetic functions

In addition to operators, Python provides built-in functions to perform arithmetic tasks. While an operator is a symbol (like `+`), a function is a named block of code that performs a specific calculation when 'called' with arguments.

Common examples include `abs()` for absolute value, `round()` for rounding numbers, and `pow()` as a functional alternative to the `**` operator.

Prompt: Give me a code example each for these function examples.

```
# Absolute value
print(f'Absolute value of -10: abs(-10) = {abs(-10)}')

# Rounding
print(f'Rounding 3.14159 to 2 decimal places: round(3.14159, 2) = {round(3.14159, 2)}')

# Power function (equivalent to 2 ** 3)
print(f'Power function: pow(2, 3) = {pow(2, 3)}')
```

Absolute value of -10: `abs(-10) = 10`
Rounding 3.14159 to 2 decimal places: `round(3.14159, 2) = 3.14`
Power function: `pow(2, 3) = 8`

✓ Practice: Call - Break - Compare

1. Use the three functions `abs()`, `round()`, and `pow()` to create one case where the result is 1, and one case where the result is larger than 10.

Start coding or [generate](#) with AI.

2. For each function, create two different calls:

- `abs()`: one with a positive number, one with a negative number.
- `round()`: one that rounds down, one that rounds up
- `pow()`: one with an exponent < 1 , one with an exponent > 1000

Start coding or [generate](#) with AI.

3. Rewrite the following expressions using functions:

```
2 ** 4 # with pow()
-7     # with abs()
3      # with round()
```

Start coding or [generate](#) with AI.

4. What happens with `round(4.5)` and `round(5.5)`? Check it out, and ask AI for an explanation. Have it add the explanation to the notebook.

Start coding or [generate](#) with AI.

Solutions:

```
# 1. Use the three functions `abs()`, `round()`, and `pow()`
# to create one case where the result is 1, and one case
# where the result is larger than 10.
```

```
print(abs(-1))
print(round(1.001))
print(pow(100,0))
```

```
print(abs(-100))
print(round(10.6))
print(pow(3,3))
```

[Show hidden output](#)

```
# abs() with a positive number, and with a negative number
# round(): one that rounds down, one that rounds up
# pow(): one with an exponent < 1, one with an exponent > 1000
print(abs(100))
print(abs(-100))
print(round(42.2))
print(round(42.9))
print(pow(4,0.5))
print(pow(2,100))
```

```
100
100
42
43
```

```
2.0
1267650600228229401496703205376
```

```
# 4. What happens with `round(4.5)` and `round(5.5)`?
# Check it out, and ask AI for an explanation.
# Have it add the explanation to the notebook.
print(round(4.5))
print(round(5.5))
```

```
4
6
```

When the fractional part is exactly `0.5`, Python's built-in `round()` function (in Python 3 and later) uses a rounding strategy called "**round half to even**" or "**banker's rounding**".

This means that if the number is precisely equidistant from two integers, it rounds to the nearest *even* integer.

- `round(4.5)` evaluates to `4` (because 4 is an even number).
- `round(5.5)` evaluates to `6` (because 6 is an even number).

This behavior is designed to minimize statistical bias when rounding a large set of numbers, which is why it's often used in financial calculations.